

ELE344 Conception et architecture de processeurs**Projet 2 : Le processeur MIPS mono-cycle***(Durée 5 semaines)*

I. Introduction

Ce projet a pour objectif de concevoir un processeur 1 cycle/instruction (monocycle). À la fin de ce projet, vous devriez comprendre l'architecture du processeur monocycle MIPS (*Microprocessor without Interlocked Pipeline Stages*) ainsi que son fonctionnement interne. Il est très important d'assimiler les bases du processeur monocycle vues en cours avant de commencer son implémentation en VHDL. Ce projet est divisé en deux parties :

1. La partie 1 a pour but d'une part de se familiariser avec les instructions de base du MIPS, et d'autre part de comprendre l'exécution d'un programme (ensemble d'instructions) dans le processeur MIPS et de générer le code machine afin de l'utiliser dans la partie 2 (section du test du processeur MIPS).
2. La deuxième partie est consacrée au design d'un processeur MIPS monocycle en VHDL, à sa vérification fonctionnelle par simulation, à sa synthèse et son test sur un carte de prototypage sur FPGA.

Une préparation doit être soumise avant le début de la deuxième séance de laboratoire (voir description de la PRÉPARATION 1). Tout retard implique une note zéro pour cet élément d'évaluation.

II. Partie 1 : Initiation au compilateur MARS de programme MIPS

Dans cette partie, vous utiliserez le simulateur MARS pour compiler et exécuter des codes assembleurs. Vous devrez comprendre le programme MIPS fournis, le compiler, l'exécuter et générer son code machine associé avec le compilateur MARS. Cette étape doit être terminée et vérifiée par l'auxiliaire de laboratoire avant la fin de la première séance de laboratoire.

Sous l'onglet « Projets » de Moodle, les différentes fonctionnalités du simulateur MARS sont brièvement décrites dans « **Procédures pour le simulateur MARS** ». Il est conseillé de comprendre et simuler des exemples de programme MIPS fournis sous l'onglet « Exercices/Exemples de code MIPS » du site Moodle du cours pour vous exercer avant d'entamer les deux étapes suivantes. Attention aux programmes 1 à 8 qui contiennent des opérations LW et SW. Leur configuration de la mémoire dans MARS doit être « Default » (Settings→Memory configuration).

Générer le code machine du programme **Code_mips_vX.docx** fourni avec l'énoncé du Projet 2 destiné à la vérification fonctionnelle du processeur MIPS. Le code machine des instructions doit être déposé à partir de l'adresse 0x0. Ce code machine sera utilisé dans la partie 2 du projet (Section : vérification par simulations). Utiliser l'outil MARS pour comprendre l'exécution de ce programme MIPS.

Une fois le code machine généré, inclure le code dans la mémoire d'instructions **imem.vhd**. Les instructions sont numérotées de 0 à n (n=TAILLE_ROM) dans **imem.vhd**.

VÉRIFICATION 1 : (1) Exporter avec MARS le code machine du programme `Code_mips_vX.docx` fourni avec l'énoncé du Projet 2 et remplir le tableau 1 du fichier `Code_mips_vX.docx` avec le code machine de chaque instruction. Faire valider ce fichier et le fichier **imem.vhd** par le chargé de laboratoire.

PRÉPARATION 1 : Compléter les tableaux 1 et 2 du fichier `Code_mips_vX.docx` et le déposer sous « Dépôt de la préparation » sous Projet 2 dans Moodle (**AVANT LE DÉBUT DE LA DEUXIÈME SÉANCE DU PROJET 2**). Le code machine de chaque instruction doit être inscrit dans le tableau 1. Le tableau 2 doit décrire le déroulement des opérations cycle par cycle, le résultat de chaque opération, le contenu du registre modifié et des valeurs observées aux ports WriteData et AluResult (address) de l'entité *top* (Figure 5).

NE PAS CHANGER L'ORDRE DES INSTRUCTIONS DANS LE FICHIER.

N.B. Les ports **WriteData (wd)** et **AluResult (address (a))** de l'entité *top* sont aussi les deux entrées de la mémoire de données (Data Memory), voir la figure 1.

III. Partie 2 : Spécifications et méthodologie de développement

L'architecture globale du processeur est donnée à la figure 1. Le processeur devra exécuter les instructions : add, sub, and, or, slt, lw, sw, beq, addi et j.

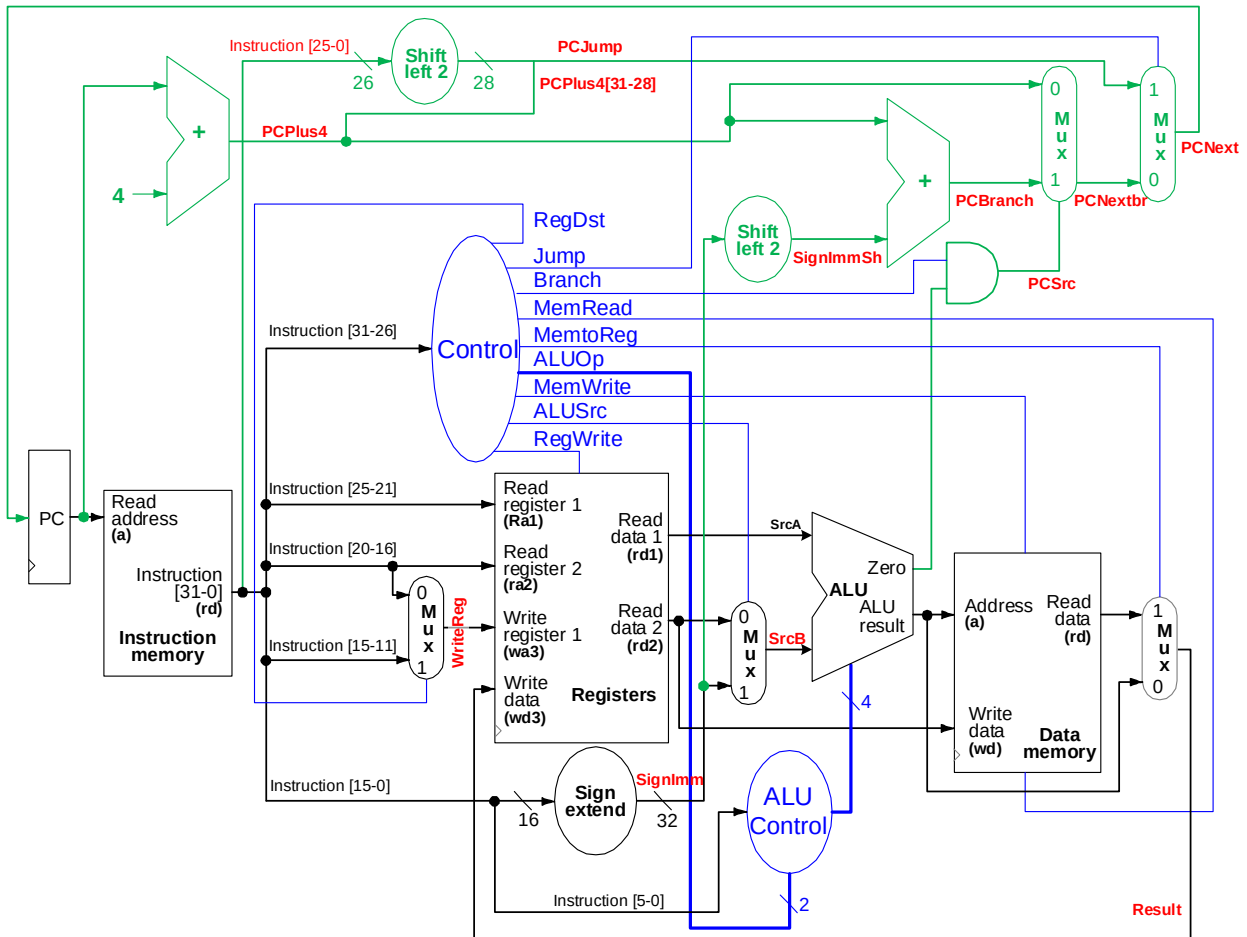


Figure 1. Architecture du processeur MIPS monocycle (1-cycle par instruction).

L'architecture du MIPS présenté à la figure 1 est divisée en plusieurs composants afin de faciliter la conception et la vérification fonctionnelle. Les composants sont : **contrôleur.vhd**, **regfile.vhd**, **ual.vhd**, **datapath.vhd**, **mips.vhd**, **imem.vhd**, **dmem.vhd** et **top.vhd**. Les composants regfile, imem et dmem sont fournis.

N.B. : Le composant imem.vhd doit être complété dans la première partie de ce projet par le nouveau code machine en se basant du code mips fournis.

Le code VHDL développé devra être synthétisé, optimisé et implémenté sur FPGA. La carte utilisée est la carte *DE10-Lite* d'Intel. Une procédure simplifiée (procedure_Quartus_vX.Pdf disponible sous l'onglet Projets sur Moodle) présente en détails les étapes nécessaires pour l'implémentation d'un circuit en VHDL sur la carte *DE10-Lite* en utilisant le logiciel *QUARTUS*.

Votre design du processeur MIPS devra respecter les **contraintes de conception suivantes** :

- 1) Le circuit doit être commandé par une et une seule horloge;
- 2) Tous les éléments synchrones doivent être sensible à la **transition montante** de l'horloge (rising_edge(clk));
- 3) Seuls les opérations ou éléments suivants sont **synchrones**, toutes les autres sont des circuits combinatoires (sans horloge) :
 - 3.1. Le registre PC (compteur de programme);
 - 3.2. L'écriture dans le banc de registres et la mémoire de donnée.
- 4) Lors de la synthèse dans Quartus, l'outil doit générer aucun message d'erreur (*error*) ou d'avertissement (*warning*) [sauf si justifié] et la fréquence d'horloge doit être au minimum 50 MHz.

A. Description des composants à concevoir

La description du processeur se divise en quatre composants: l'unité de contrôle (Controller), le chemin de données (Datapath), le processeur MIPS (MIPS), et pour l'intégration du processeur dans le FPGA, le top.

A.1. Composant : Unité de contrôle

Les signaux d'entrée-sortie de l'unité de contrôle (*Controller*) sont montrés au tableau 2.

Tableau 2 Signaux entrées-sorties de l'unité de contrôle.

Unité de contrôle					
OP	(5:0)	Entrée		AluSrc	Sortie
Funct	(5:0)	Entrée		RegDst	Sortie
Branch		Sortie		RegWrite	Sortie
MemtoReg		Sortie		Jump	Sortie
MemRead		Sortie		AluControl	(3:0) Sortie
MemWrite		Sortie			

Cette unité génère les signaux de contrôle en se basant sur les champs *opcode* et *Funct* de l'instruction (voir entité ci-dessous). La figure 2 montre la structure interne de l'unité de contrôle.

```

ENTITY controller IS
PORT (
  OP, Funct: IN std_logic_vector(5 downto 0);
  MemtoReg, MemWrite, MemRead, Branch, AluSrc,
  RegDst, RegWrite, Jump : OUT std_logic;
  AluControl : OUT std_logic_vector(3 downto 0));
END Controller;
  
```

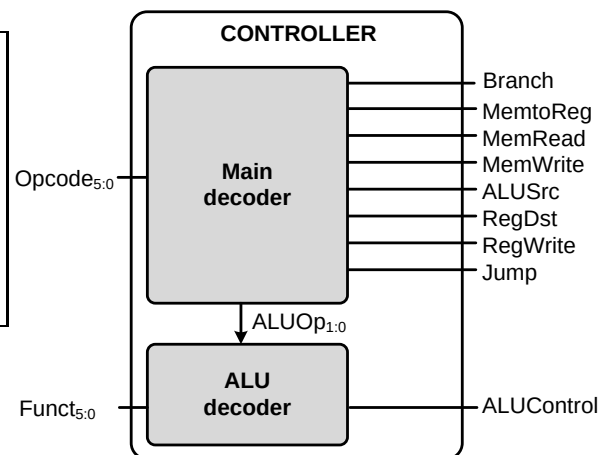


Figure 2. Structure interne de l'unité de contrôle.

Le *Main decoder* (module *Control* de la figure 1) reçoit en entrée le code opération (OP, noté également Opcode) et génère en sortie les signaux de contrôle représentés par la table de vérité du tableau 3.

Tableau 3 Table de vérité du Main decoder

Instr	OPCode	RegWrite	RegDst	ALUSrc	Branch	MemRead	MemWrite	MemtoReg	ALUOp	Jump	Code Hexa
R-Type	0000000	1	1	0	0	0	0	0	10	0	304
Lw	100011	1	0	1	0	1	0	1	00	0	2A8
Sw	101011	0	-	1	0	0	1	-	00	0	090
Beq	000100	0	-	0	1	0	0	-	01	0	042
Addi	001000	1	0	1	0	0	0	0	00	0	280
J	000010	0	-	-	-	0	0	-	--	1	001

En VHDL, un « - » (don't care) est utilisé dans les expressions logiques (comme en synthèse ou en tables de vérité) pour indiquer qu'une valeur peut être soit 0 soit 1.

L'*ALU decoder* (module *ALU control* de la figure 1) utilise les signaux *ALUOp* et le champ *Funct* de l'instruction pour générer la sortie *ALUControl*, telle que montré au tableau 4.

Tableau 4 Table de vérité du ALU decoder

ALUOp	Funct (5:0)	ALUControl*
00	-	0010 (add)
01	-	0110 (sub)
10	100000 (add)	0010 (add)
	100010 (sub)	0110 (sub)
	100100 (and)	0000 (and)
	100101 (or)	0001 (or)
	101010 (slt)	0111 (slt)
11		n/a

Vous devez coder le *Main decoder* et l'*ALU Decoder* dans des process VHDL combinatoires différents dans l'architecture de l'unité de contrôle (*Controller*).

Simulation de l'entité *controller*

Valider la fonctionnalité de l'entité *controller* et son architecture. Il doit injecter séquentiellement tous les *OPCodes* identifiées au tableau 3 et générer les valeurs de chaque signal de contrôle.

VÉRIFICATION 2 : Lorsque votre simulation de votre design *controller* est terminée, **AVANT LA FIN DE LA 3^e SÉANCE DE LABORATOIRE DU PROJET 2**, faire valider votre simulation et le bon fonctionnement du design par votre chargé de laboratoire. Les résultats de simulation doivent être présentés sous forme de forme d'onde (Wave) dans Modelsim.

Vous pouvez faire un banc d'essai (testbench) pour le contrôleur ou simplement injecter les valeurs de *OPCode* et *Funct* en utilisant la commande **force** dans un fichier .do. Vous devez tester les 6 valeurs de « *OPCode* » et 5 valeurs de « *Funct* » décrites dans les tableaux 4 et 5.

* ALUControl (3) n'est pas utilisé dans tout le lab. L'étudiant-e pourra plus tard ajouter de nouvelles fonctionnalités au présent processeur.

L'architecture du *Datapath* est présentée à la figure 3 où les signaux d'entrées sont en bleu et les signaux de sorties en rouge, tel que résumés au tableau 5. **Vous devez écrire le code VHDL du datapath en se basant de l'architecture suivante.**

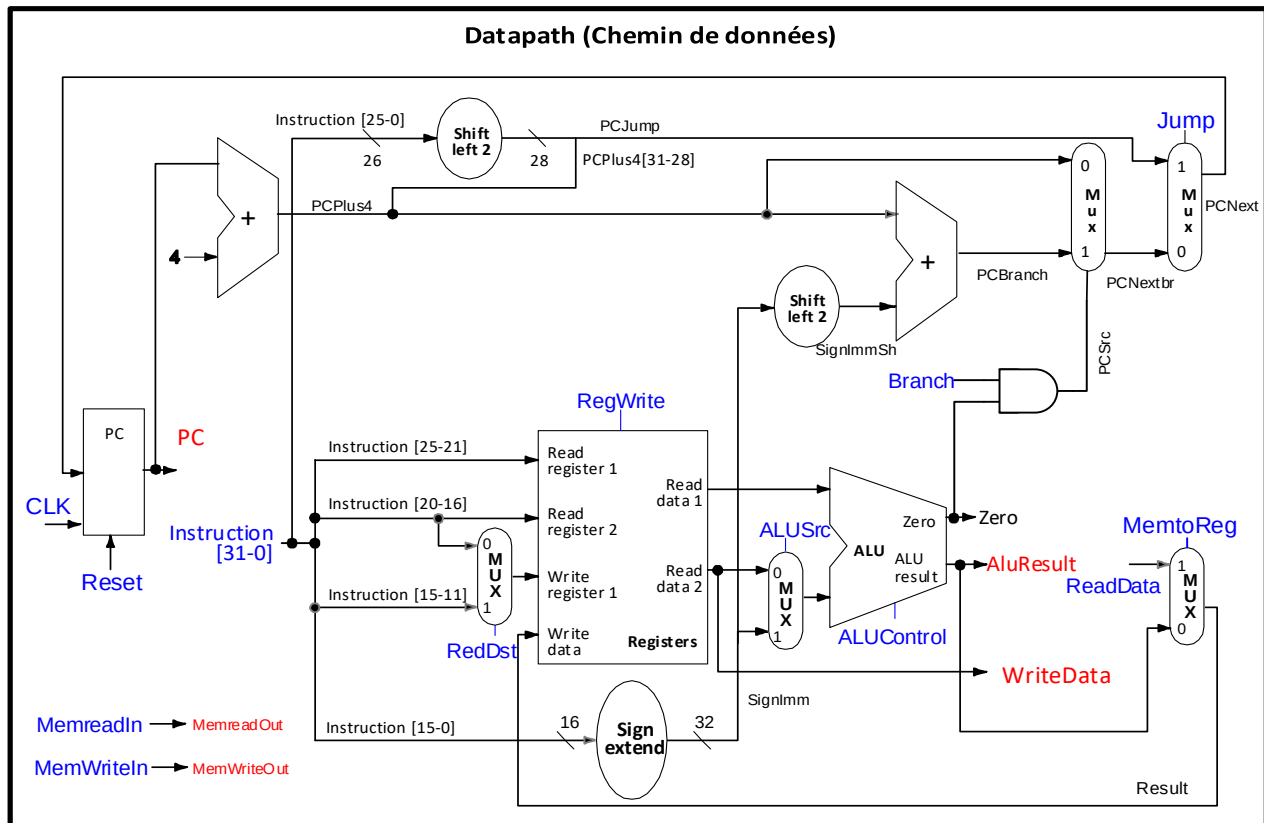


Figure 3 : Architecture du Datapath (chemin de données).

Tableau 5 Signaux entrées-sorties du datapath

Datapath (chemin de données)						
Clk		Entrée		MemWriteIn		Entrée
Reset		Entrée		AluControl	(3:0)	Entrée
MemtoReg		Entrée		Instruction	(31:0)	Entrée
Branch		Entrée		ReadData	(31:0)	Entrée
AluSrc		Entrée		MemReadOut		Sortie
RegDst		Entrée		MemWriteOut		Sortie
RegWrite		Entrée		PC	(31:0)	Sortie
Jump		Entrée		AluResult	(31:0)	Sortie
MemReadIn		Entrée		WriteData	(31:0)	Sortie

Le chemin de données *Datapath* est composé de trois groupes de logique : la logique pour le calcul du PC (nommé PCpath), la logique du banc de registres et la logique de l'ALU. Les opérations logiques pour chacun de ces groupes et les contraintes pour leur réalisation sont décrites aux tableaux 5, 6 et 7 respectivement. Les opérations doivent être codées dans **un nombre minimum** de process VHDL et d'énoncés concurrents VHDL dans l'architecture du *Datapath*, et en utilisant le package `numeric_std` de l'IEEE pour les opérations arithmétiques.

Vous devez également instancier dans le Datapath les composants définis dans **regfile.vhd** et **alu.vhd** (fait au projet 1) en prenant soin de définir des signaux internes significatifs pour faire leurs connexions.

Tableau 6 Logique pour le calcul du PC

Signal	Description de l'opération	Réalisation RTL
PCPlus4	PC + 4	opérateur « + »
PCBranch	PCPlus4 + SignImmSh	opérateur « + »
PCJump	PCPlus4 (31:28) & Instruction (25:0) & "00"	Simple concaténation
PC	bascule D synchrone avec remise à zéro asynchrone (clear), prenant comme entrée clk et PCNext	Process synchrone
SignImmSh	SignImm est décalé de 2 bit à gauche	Connexion décalée
PCNextbr	un multiplexeur contrôlé par <i>PCSrc</i>	CASE ou IF imbriqué
PCNext	un multiplexeur contrôlé par <i>Jump</i>	CASE ou IF imbriqué

Tableau 7 Logique du banc de registres

Signal/composant	Description de l'opération	Réalisation RTL
Banc de registres	prend en entrée les adresses ra1, ra2, wa et la donnée wd. Fait la lecture en mémoire. Il retourne en sortie rd1, rd2. La banque de registres est contrôlé par <i>RegWrite</i>	RegFile
WriteReg	un multiplexeur contrôlé par <i>RegDst</i> . Permet de sélectionner l'adresse du registre pour écrire le résultat de l'opération addi par exemple	CASE ou IF imbriqué
Result	un multiplexeur contrôlé par MemtoReg (sortie provenant de l'ALU ou de la mémoire)	CASE ou IF imbriqué
SignImm	extension de signe (16 à 32 bits)	SignExtend

Tableau 8 Logique de l'ALU

Signal/composant	Description de l'opération	Réalisation RTL
SrcB	un multiplexeur contrôlé par <i>AluSrc</i>	CASE ou IF imbriqué
ALU 32bits	déjà développée dans le projet précédant	ALU_32

A.3. Composant : processeur MIPS

Écrire une entité appelée *MIPS* avec son architecture qui instancie les deux composants *Controller* et *Datapath*. La hiérarchie du design VHDL est présentée à la figure 4.

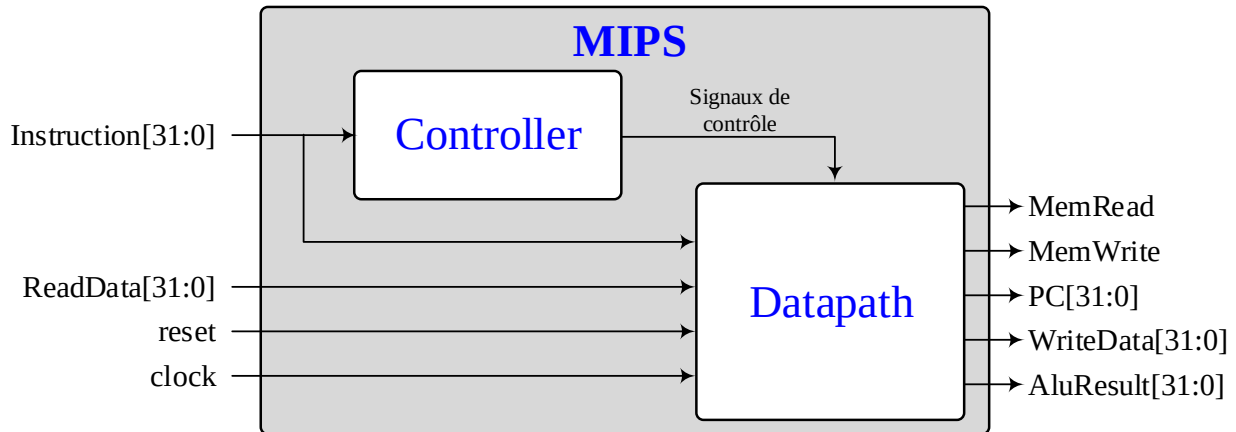


Figure 4. Le composant MIPS.

A.4. Composant : TOP

Écrire une entité appelée *top* et son architecture qui instancie les composants *MIPS* et les mémoires d'instructions (*imem*) et de données (*dmem*). Ces deux mémoires sont décrites respectivement dans les fichiers *imem.vhd* et *dmem.vhd* fournis avec l'énoncé du projet. Ces descriptions simplifiées de mémoires peuvent être simulées et inférées directement dans le FPGA. L'entité *top* et son architecture interne doivent être semblable à celles représentées à la figure 5. Cette entité *top* doit être validée avec le programme MIPS tel que défini à la section C.

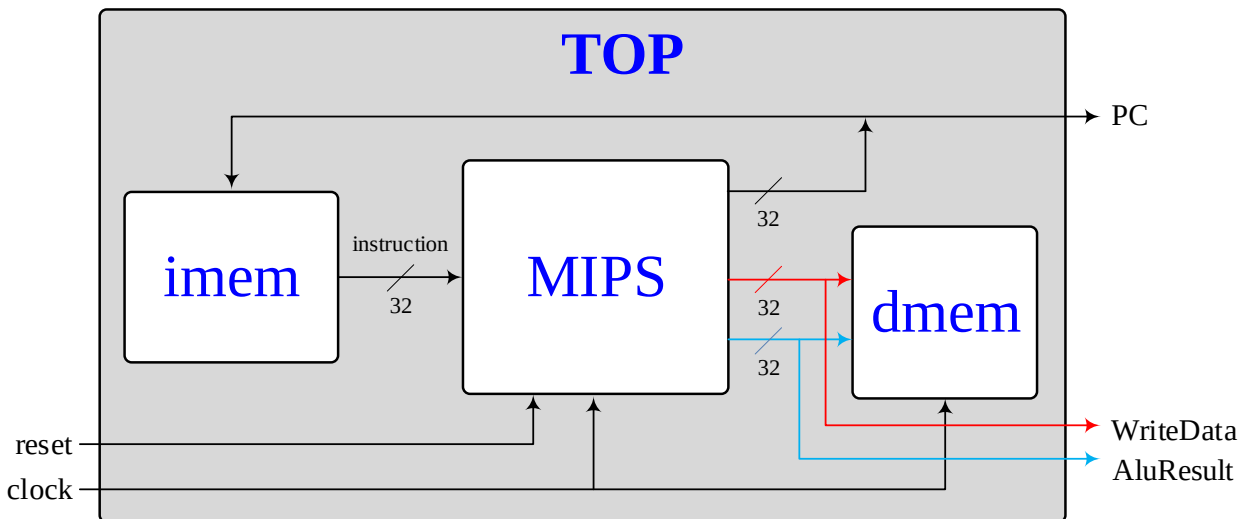


Figure 5. Le composant top.

N.B Les signaux internes du module Top ne sont pas représentés dans la Figure 5.

B. Vérification fonctionnelle par simulation

Il est conseillé de vérifier la fonctionnalité par simulation de chaque composant réalisé. Ces vérifications impliquent l'existence d'un banc d'essai VHDL pour chaque nouveau module. Une fois les composants vérifiés comme fonctionnels et synthétisable en respect des contraintes, vous allez les assembler dans une entité TOP et vérifier l'ensemble par simulation.

Le processeur interagit avec 2 mémoires : la mémoire ROM d'instructions (`imem.vhd`) et la mémoire de données (`dmem.vhd`) où le processeur mémorise les données. Les instructions à exécuter et à utiliser pour vérifier par simulation la fonctionnalité du processeur MIPS sont définies dans le fichier `imem.vhd`, que vous avez créé à la partie 1 du projet 2.

Vous devez mettre à jour le fichier **imem.vhd** avec le code binaire généré avec MARS. Le code binaire des instructions devrait être validé avec le chargé de laboratoire lors la vérification 1 avant de passer à la prochaine section. Si c'est déjà fait, vous pouvez passer à la section suivante.

B.1. Simulation de l'entité *top*

Un banc d'essai VHDL pour l'entité *top* est inutile car le code assemblé est déjà dans la ROM `imem.vhd`. Simulez l'entité *top* avec ModelSim en exécutant les commandes suivantes (attention à la polarité du signal `clear` selon votre implémentation) :

```
vsim Top
add wave -r *
force clk 1,0 10 ns -repeat 20 ns
force reset 1,0 15 ns
run 500 ns
```

Comparer les valeurs obtenues avec les valeurs théoriques et déboguer.

VÉRIFICATION 3 : Lorsque votre vérification du bon fonctionnement de votre design par simulation avec ModelSim est terminée, faire valider votre simulation par votre chargé de laboratoire. La simulation doit être claire, lisible et doit afficher dans une fenêtre « wave » les signaux CLK, RESET, PC (9:2), DATAADDRESS et WRITEDATA de la mémoire de données, dans cet ordre. La simulation doit exécuter avec succès tout le programme.

C. Synthèse de `top.vhd` dans Quartus et optimisation

Un des objectifs de conception est que votre design doit opérer à une fréquence d'horloge minimale de $F_{\min} = 50 \text{ MHz}$ vers le FPGA de la carte de prototypage DE10-Lite d'Intel.

Faire la synthèse du composant **top.vhd** avec Quartus. Modifier le fichier **top_fpga.sdc** pour spécifier la période d'horloge et l'inclure dans Quartus avec vos fichiers VHDL.

Vérifier les rapports de synthèse, entre autres la fréquence maximale, les ressources utilisées, les délais, leurs chemins. Résoudre les erreurs de synthèse et valider les avertissement (*warnings*) (p. ex. doit contenir aucun latch). Corriger votre design pour respecter la contrainte de fréquence d'horloge, par exemple par l'optimisation des chemins critiques, au besoin.

VÉRIFICATION 4 : Lorsque la synthèse sera terminée et vérifiée, ouvrir tous vos fichiers VHDL et faire valider par votre chargé de laboratoire. Le *slack* doit être positif et la fréquence maximale d'opération F_{clk} supérieure à $F_{\min}$.

Le barème de correction sous la section « Description HDL pour la synthèse » du document `grille_evaluation_fichier_VHDL.pdf` sera entre autres utilisé.

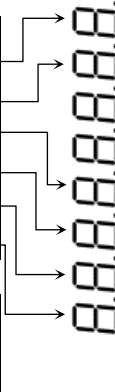
D. Implémentation et vérification sur la carte de prototypage DE10-Lite d'Intel

Une fois la vérification par simulation et la synthèse du top terminées, vous devez migrer vers le logiciel Quartus pour (1) créer un nouveau projet et y inclure vos fichiers VHDL avec **top_fpga.vhd**, (2) faire et vérifier la synthèse vers le FPGA et (3) vérifier le bon fonctionnement de votre circuit sur la carte de prototypage *DE10-Lite* d'Intel. Suivez les instructions décrites dans le document *procedure_Quartus_vX.pdf*. **Avant la fin de la dernière séance, vous devez faire valider le fonctionnement de votre design sur la carte de prototypage par le chargé de laboratoire.**

D.1. Entité : top_fpga

Afin de pouvoir visualiser les résultats des ports de sortie de l'entité *top*, on affichera respectivement les valeurs du PC, DataAddress et WriteData de la mémoire de données sur les afficheurs 7-segments de la carte *DE10-Lite* : HEX0, HEX1, HEX2, HEX3, HEX4 et HEX5. On associera chaque 4-bits à un décodeur 7-segments connectés, tel que décrit au tableau 9.

Tableau 9 Connexions du top aux afficheurs 7-segments et aux commutateurs sur la carte DE10

Ports de sortie de l'instance top	Afficheur 7-segment	
PC(5 downto 2)	HEX0	
PC(9 downto 6)	HEX1	
DataAddress(3 downto 0)	HEX2	
DataAddress(7 downto 4)	HEX3	
WriteData(3 downto 0)	HEX4	
WriteData(7 downto 4)	HEX5	
Ports d'entrée de l'instance top	Commutateurs	
Reset	KEY[0]	
Clock	KEY[1]	

Deux fichiers **top_fpga.vhd** et **dec7seg.vhd** sont fournis. L'entité *top_fpga* instancie le composant *top* et les 6 décodeurs 7-segments. Les ports du *top_fpga* ont les mêmes noms que ceux des pins définies pour la carte *DE10-Lite*.

D.2. Vérification de la fonctionnalité sur la carte de prototypage DE10-Lite

Chargez votre design sur la carte selon la procédure décrite dans le fichier **procédure d'utilisation de l'outil Quartus.pdf**, régler les messages les erreurs, et faites la vérification de la fonctionnalité. Inspectez les rapports de synthèse de Quartus pour extraire la fréquence maximale d'opération et l'espace occupée sur le FPGA.

La vérification du fonctionnement et de la synthèse doit se faire avec le chargé de laboratoire avant la fin de la 5^e séance de laboratoire de ce projet.

VÉRIFICATION 5 : Lorsque votre vérification du bon fonctionnement de votre design est terminée sur la carte *DE10-Lite*, faire valider par votre chargé de laboratoire, et ce **AVANT LA FIN DE LA 5^e SÉANCE DE LABORATOIRE**.

E. Bonus (facultatif)

Si réussi, ce bonus de 10 % s'ajoute à l'ensemble des projets. Notez que la note finale pour l'ensemble des projets ne peut pas dépasser 100 %.

Créer un programme MIPS qui fait l'équivalent d'un chronomètre numérique en modifiant le contenu de quatre adresses en mémoire de données :

- Le contenu de l'adresse 0x0 incrémente à une cadence de 0.1 seconde et retombe à zéro après la valeur 9 (0,1,2,3,4,5,6,7,8,9,0,...);
- Le contenu de l'adresse 0x4 incrémente à une cadence de 1 seconde et retombe à zéro après la valeur 9;
- Le contenu de l'adresse 0x8 incrémente à une cadence de 10 secondes et retombe à zéro après la valeur 5;
- Le contenu de l'adresse 0xC incrémente à une cadence de 60 secondes (minute) et retombe à zéro après la valeur 9.

À chaque modification d'une de ces adresses en mémoire de données, la valeur est affichée à son afficheur HEX respectif. L'architecture du top_fpga devra être modifiée pour capter les données écrites aux adresses respectives de la mémoire de données. Notez que l'horloge du processeur est connectée au cristal de la carte, avec une fréquence de 50 MHz. Il est défendu d'instancier des compteurs en VHDL.

Faire une démonstration au chargé de laboratoire du bon fonctionnement de votre design avant la fin du projet 2. Le design doit être 100 % fonctionnel pour obtenir le bonus (tout ou rien).

F. Documents à remettre (Un seul fichier zip déposé sous l'onglet « Dépôt du rapport » sous le Projet2 du site Moodle du cours)

À la complétion du projet, remettre les documents suivants dans un et un seul fichier zip et le déposer sous l'onglet « Dépôt du projet 2 » du site Moodle du cours (ena.etsmtl.ca) avec comme nom : ELE344_PROJET2_KABI_GOUM.zip

où KABI et GOUM sont les lettres du code permanent des membres de l'équipe.

Le fichier **Zip** doit contenir :

- 1) Les fichiers **VHDL**
- 2) Les fichiers **.do** (si utilisé)
- 3) Pour un projet non fonctionnel à 100 % selon les vérifications 2 et 3, un rapport obligatoirement nommé : **Rapport_Projet2_KABI_GOUM.docx**. Les détails sont présentés à la section « Détails du rapport ». Utilisez le fichier rapport_gabarit.docx pour la rédaction de ce rapport.

F.1. Sections du rapport pour un projet non fonctionnel

Un rapport devra être remis au cas où le design ne serait pas complètement fonctionnel lors de la vérification 2 ou 3 et doit contenir quatre sections. Ce rapport permettra d'obtenir jusqu'à 75 % des points non obtenus aux Vérifications 2, 3 et 4. Ce rapport doit contenir les sections suivantes :

Introduction et objectifs du projet

Description du processeur MIPS monocycle:

Descriptions détaillées du fonctionnement du MIPS et le rôle de ses composantes (PC, mémoire, registre, etc.)

Localisation et causes d'erreurs :

Explications des causes ayant pu mener aux erreurs de conception s'étant produites dans votre design. Ces explications doivent être appuyer par des chronogrammes des parties ne fonctionnant pas correctement et des rapports de synthèse (s'il y a lieu). Ces chronogrammes sous forme de captures d'image doivent inclure :

- ✓ Les signaux représentés en hexadécimal.
- ✓ Les formes d'ondes doivent inclure entre autres les signaux suivants et dans cet ordre : CLK, Reset, PC, WRITEDATA de la mémoire de données, AluResult.

Discussion et amélioration possibles:

Discussion mettant en lien les causes d'erreurs et les améliorations pouvant être apportées. Les pistes de solutions possibles devront y être expliquées.

Annexe 1 Documents fournis sous Projet 2 dans Moodle

Fichier	Description	Action
Projet2_vX.pdf	Énoncé du projet	à lire
imem.vhd top_fpga.vhd	Fichiers VHDL du processeur MIPS	à compléter
regfile.vhd dmem.vhd dec7seg.vhd	Fichiers VHDL du processeur MIPS fournis et complets	aucune
top_fpga.sdc	Fichier de contraintes pour Quartus	à adapter
exemple.do	Exemple simple de fichier .do	à adapter
Code_mips_vX.docx	Exemple du code assembleur à tester avec le processeur MIPS réalisé dans ce projet.	à adapter
Requis_programmes_MIPS.docx	Énoncé du problème à résoudre pour la vérification 1 (Étape 3)	à lire
Gabarit_rapport.docx	Gabarit pour la rédaction de votre rapport	à compléter

Annexe 2 Barème d'évaluation du Projet 2

	Critères	Description	Note	Poids (%)
Partie 1	Vérification 1	Code machine du programme Code_mips_vX.docx		5
	Préparation 1	Tableaux 1 et 2 du fichier Code_mips_vX complétés et remis sous Moodle avant le début de la deuxième séance de laboratoire du projet		10
Partie 2	Vérification 2	Résultats de simulation du controller (3 ^e séance)		5
	Vérification 3	Résultats de simulation du top		35
	Vérification 4	Rapports de synthèse du top.vhd avec Quartus (fréquence maximale, ressources utilisées, slack).		15
	Vérification 5	Fonctionnement du design sur la carte DE10-Lite (avant la fin de la 5 ^e séance)		10
	Vérification 6	Document zip déposé sur Moodle		20
		Rapport du projet : Rapport_Projet2_XXX.docx*		
		Fichiers VHDL** et .do		
Total :				100
Bonus*** :				10

* Voir la description du contenu à la Section F.1. Ce rapport permet d'obtenir jusqu'à 75 % des points non obtenus aux Vérifications 2, 3 et 4.

** La forme et le contenu des fichiers VHDL sont évalués selon les barèmes spécifiés dans le fichier *grille_evaluation_fichier_VHDL.pdf*

*** Si réussi, ce bonus s'ajoute à l'ensemble des projets. Notez que la note pour l'ensemble des projets ne peut pas dépasser 100 % de leur poids dans la note finale de l'étudiant.